

Functional Dependency in DBMS

A functional dependency occurs when the value of one attribute (or a set of attributes) uniquely determines the value of another attribute. This relationship is denoted as:

$X \rightarrow Y$

Here, **X** is the determinant, and **Y** is the dependent attribute. This means that for each unique value of X, there is precisely one corresponding value of Y.

Consider a table named Students with the following attributes:

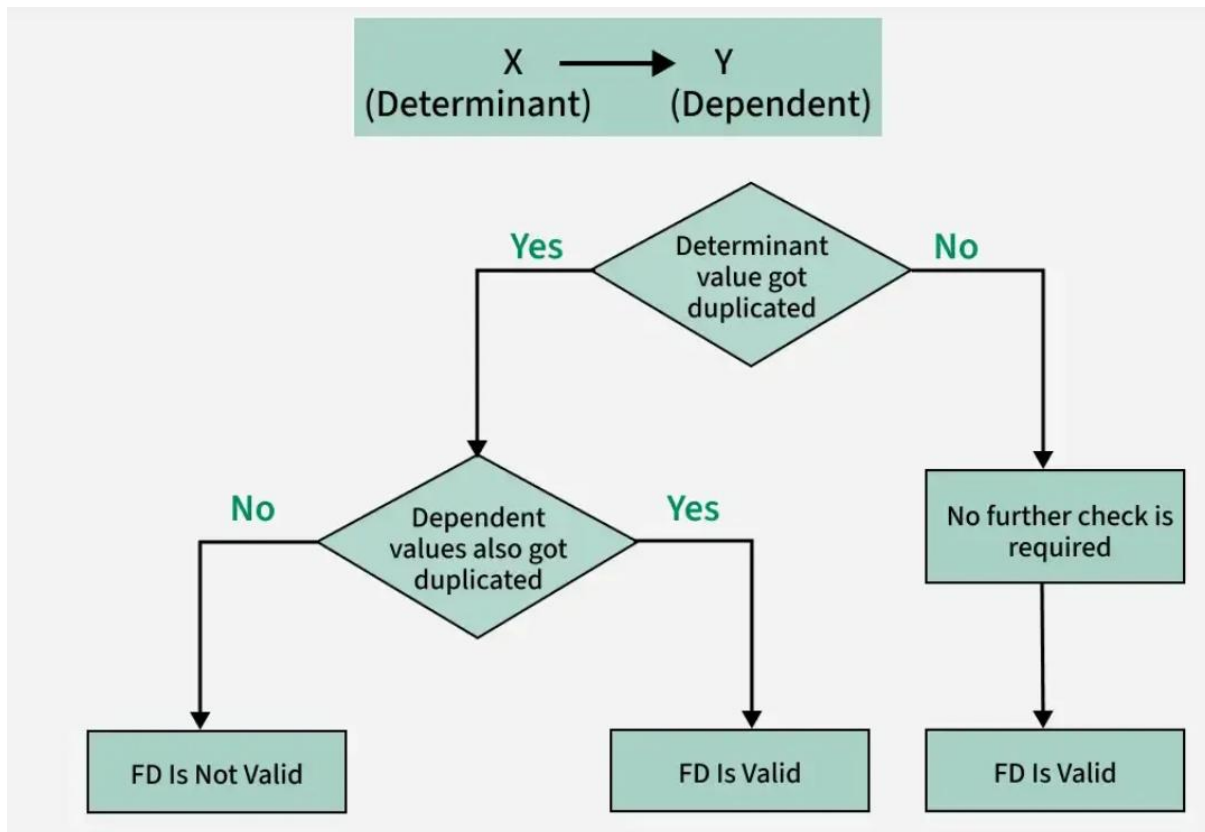
StudentID	StudentName	StudentAge
101	Rahul	23
102	Ankit	22
103	Aditya	22
104	Sahil	24
105	Ankit	23

The above table has the following functional dependencies

StudentID -> StudentName

StudentID -> StudentAge

Note that the functional dependencies StudentName -> StudentAge or StudentAge -> StudentName **Do not hold**.



How to represent functional dependency in DBMS?

- Functional dependency is expressed in the form of equations. For example, if we have an employee record with fields "EmployeeID", "FirstName" and "LastName" we can specify the function as follows:

EmployeeID -> FirstName, LastName

- To represent functional dependency in DBMS has two main features: left (LHS) and right (RHS) of the arrow (->).
- For example, if we have a table with attributes "X", "Y" and "Z" and the attribute "X" can determine the value of the attributes "Y" and "Z".

$X \rightarrow Y, Z$

- This symbol indicates that the value in attributes "X" determines the values in attributes "Y" and "Z". So if you know the value of "X", you can also determine the value of "Y" and "Z".

Benefits of Functional Dependency in DBMS

The concept of normalization is based on functional dependencies. Using functional dependencies, we break a table in multiple tables that helps us in preventing duplicate data and hence Improves Data Quality, less errors and better database design.

Types of Functional dependencies in DBMS

A functional dependency occurs when one attribute uniquely determines another attribute within a relation. There are the following types of functional dependencies.

1. Trivial Functional Dependency

In Trivial Functional Dependency, a dependent is always a subset of the determinant. i.e., If $X \rightarrow Y$ and Y is the subset of X , then it is called a trivial functional dependency.

Symbolically: $A \rightarrow B$ is a trivial functional dependency if B is a subset of A .

The following dependencies are also trivial: $A \rightarrow A$ & $B \rightarrow B$

Example 1 :

- $ABC \rightarrow AB$
- $ABC \rightarrow A$
- $ABC \rightarrow ABC$

Example 2:

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent name is a subset of determinant set $\{\text{roll_no}, \text{name}\}$. Similarly, $\text{roll_no} \rightarrow \text{roll_no}$ is also an example of trivial functional dependency.

2. Non-trivial Functional Dependency

In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant. i.e. If $X \rightarrow Y$ and Y is not a subset of X , then it is called Non-trivial functional dependency.

Example 1 :

- Id -> Name
- Name -> DOB

Example 2:

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, roll_no $\rightarrow$ name is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll_no. Similarly, {roll_no, name} $\rightarrow$ age is also a non-trivial functional dependency, since age is not a subset of {roll_no, name}

3. Multivalued Functional Dependency

In Multivalued functional dependency, entities of the dependent set are not dependent on each other. i.e. If $a \twoheadrightarrow \{b, c\}$ and there exists no functional dependency between b and c, then it is called a multivalued functional dependency.

Example:

bike_model	color	accessory
tu1001	Black	Helmet
tu1001	Black	Gloves
tu1001	Red	Helmet
tu1001	Red	Gloves

In this table:

- X = bike_model

- Y = color
- Z = accessory

For each bike model (bike_model):

1. There is a group of colors (color) and a group of accessories (accessory).
2. The colors do not depend on the accessory, and the accessories do not depend on the colors. They are independent of each other.
3. The sets of color and accessory are linked only to bike_model.

That's what makes it a multivalued dependency.

In this case, these two columns (color and accessory) are said to be multivalued dependent on bike_model. These dependencies can be represented like this:

bike_model $\twoheadrightarrow$ color

bike_model $\twoheadrightarrow$ accessory

4. Transitive Functional Dependency

In transitive functional dependency, dependent is indirectly dependent on determinant. i.e. If $a \rightarrow b$ & $b \rightarrow c$, then according to axiom of transitivity, $a \rightarrow c$. This is a transitive functional dependency.

Example:

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

Here, enrol_no $\rightarrow$ dept and dept $\rightarrow$ building_no. Hence, according to the axiom of transitivity, enrol_no $\rightarrow$ building_no is a valid functional dependency. This is an indirect functional dependency, hence called Transitive functional dependency.

Transitive Dependency

- If the value of a non-primary attribute can be defined using another non-primary attribute then it is called a transitive dependency.
- When any attribute does not require primary key and can easily get value using another non-primary attribute then it is called as **Transitive Dependency**.

Example

Let's take an example, we have a table where we have columns of student roll number, name, city where student live, and zip-code of city .

Table

roll_no	name	city	zip-code
1	abc	pune	411044
2	jkl	mumbai	400001
3	uvw	pune	411044
4	xyz	delhi	110001

- Here the primary key is roll_no, but we can identify the city using zip-code where both city and zip-code are non-primary attributes.
- So here **roll_no → city** and **city → zip-code** eventually resulting into **roll_no → zip-code**. so we can find a non-primary attribute using another non-primary attribute. For example, roll-no = 1 has city=pune and city=pune will have zip-code=411044. So wherever city is pune , zip-code will be 411044
- This type of functional Dependency is known as **Transitive Dependency**.

Partial Dependency

- If the value of a non-primary attribute can be defined using part of the primary key then it is called a **partial dependency**. Partial dependency occurs when primary key is formed using more than one attribute. This type of key also called as [composite key](#).
- In below given example, the primary key is formed using **roll_no + sub_id** which can also be called as composite key. When composite key is present and one of the non-primary attribute can be dependent on part of the primary key instead of whole primary key then it is called as Partial Dependency.

Example

Let's take an example, we have a table where we have columns of student roll number, subject ID, sub name, and marks obtained.

Table

roll_no	sub_id	sub_name	sub_mark
1	121	Science	80
1	131	Math	65
2	131	Math	95
2	141	English	75

- Here primary key will be **roll_no + sub_id** because multiple roll_no can have the same sub_id and the same roll_no can have multiple sub_id. In the given example, roll_no 1 has two sub_id i.e. 121 and 131 where as sub_id 131 has two roll_no 1 and 2. So here primary key will be **roll_no + sub_id**.
- But we do have another column of sub_name and the value of sub_name can be easily obtained by only sub_id which is part of the primary key. For example, sub_id = 131 will have the sub_name = 'math' here we required only partial primary key i.e. sub_id.
- This type of functional Dependency is known as **Partial Dependency**.

Full Dependency

- If all attributes of the primary key are required for the identifying value of a non-primary attribute then it is known as **Full Dependency**.
- When all non-primary attribute are dependent on whole primary key and they cannot be get defined using only partial part of primary key then it is called as **Full Dependency**.
- If the dependency is **non-partial dependency** then it can be called as Full Dependency also.

Example

Let's take an example, we have a table where we have columns of student roll number, subject ID, and marks obtained.

Table

roll_no	sub_id	marks
1	121	80
1	131	65
2	131	95
2	141	75

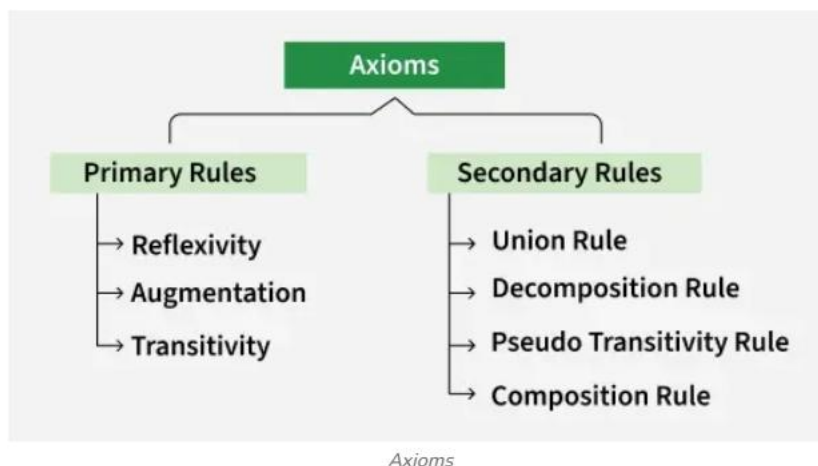
- Here the primary key is roll_no+ sub_id. If we want a mark of any student, we require both roll_no and sub_id. We cannot obtain marks based on one attribute from the primary key.
- If we want to know the marks of sub_id=131 there will be two records and ambiguity will be created. If we take roll_id=1 there will be two records with the same roll number and ambiguity will be created here. This ambiguity will be solved using full attributes of the primary key i.e. roll_no + sub_id. So we required the full attributes of the primary key.
- This type of functional Dependency is known as **Full Dependency**.

Armstrong's Axioms in Functional Dependency in DBMS

Armstrong's Axioms refer to a set of inference rules that are used to test the logical implication of functional dependencies.

- Given a set of functional dependencies F , the closure of F (denoted as F^+) is the set of all functional dependencies logically implied by F .
- Armstrong's Axioms, when applied repeatedly, help generate the closure of functional dependencies.
- These axioms are fundamental in determining functional dependencies in databases and are used to derive conclusions about the relationships between attributes.

The following are different axioms.



Axioms

- **Axiom of Reflexivity:** If A is a set of attributes and B is a subset of A , then A holds B . If $B \subseteq A$ then $A \rightarrow B$. This property is trivial property.
- **Axiom of Augmentation:** If $A \rightarrow B$ holds and Y is the attribute set, then $AY \rightarrow BY$ also holds. That is adding attributes to dependencies, does not change the basic dependencies. If $A \rightarrow B$, then $AC \rightarrow BC$ for any C .
- **Axiom of Transitivity:** Same as the transitive rule in algebra, if $A \rightarrow B$ holds and $B \rightarrow C$ holds, then $A \rightarrow C$ also holds. In the dependency $A \rightarrow B$ we say that A functionally determines B . For example, if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Example: Let's assume the following functional dependencies:

$\{A\} \rightarrow \{B\}$

$\{B\} \rightarrow \{C\}$

$\{A, C\} \rightarrow \{D\}$

1. **Reflexivity:** Since any set of attributes determines its subset, we can immediately infer the following:

- $\{A\} \rightarrow \{A\}$ (A set always determines itself).
- $\{B\} \rightarrow \{B\}$.
- $\{A, C\} \rightarrow \{A\}$.

2. **Augmentation:** If we know that $\{A\} \rightarrow \{B\}$, we can add the same attribute (or set of attributes) to both sides:

- From $\{A\} \rightarrow \{B\}$, we can augment both sides with $\{C\}$: $\{A, C\} \rightarrow \{B, C\}$.
- From $\{B\} \rightarrow \{C\}$, we can augment both sides with $\{A\}$: $\{A, B\} \rightarrow \{A, C\}$.

3. **Transitivity:** If we know $\{A\} \rightarrow \{B\}$ and $\{B\} \rightarrow \{C\}$, we can infer that:

- $\{A\} \rightarrow \{C\}$ (Using transitivity: $\{A\} \rightarrow \{B\}$ and $\{B\} \rightarrow \{C\}$).

Although Armstrong's axioms are sound and complete, there are additional rules for functional dependencies that are derived from them. These rules are introduced to simplify operations and make the process easier.

Secondary Rules

These rules can be derived from the above axioms.

- **Union:** If $A \rightarrow B$ holds and $A \rightarrow C$ holds, then $A \rightarrow BC$ holds. If $X \rightarrow Y$ and $X \rightarrow Z$ then $X \rightarrow YZ$.
- **Composition:** If $A \rightarrow B$ and $X \rightarrow Y$ hold, then $AX \rightarrow BY$ holds.
- **Decomposition:** If $A \rightarrow BC$ holds then $A \rightarrow B$ and $A \rightarrow C$ hold.
If $X \rightarrow YZ$ then $X \rightarrow Y$ and $X \rightarrow Z$.
- **Pseudo Transitivity:** If $A \rightarrow B$ holds and $BC \rightarrow D$ holds, then $AC \rightarrow D$ holds.
If $X \rightarrow Y$ and $YZ \rightarrow W$ then $XZ \rightarrow W$.

Example: Let's assume we have the following functional dependencies in a relation schema:

$\{A\} \rightarrow \{B\}$

$\{A\} \rightarrow \{C\}$

$\{X\} \rightarrow \{Y\}$

$\{Y, Z\} \rightarrow \{W\}$

Now, let's apply the **Secondary Rules** to derive new functional dependencies.

1. **Union Rule:** If $A \rightarrow B$ and $A \rightarrow C$, then by the **Union Rule**, we can infer:

- $A \rightarrow BC$ This means if A determines B and C individually, it also determines their combination, BC .

2. **Composition Rule:** Since $A \rightarrow B$ and $X \rightarrow Y$ hold, then by the **Composition Rule**, we can infer:

- $AX \rightarrow BY$

3. **Decomposition Rule:** Using the dependency $A \rightarrow BC$ (derived in step 1), the **Decomposition Rule** allows us to break it back down to infer:

- $A \rightarrow B$ and $A \rightarrow C$

4. **Pseudo Transitivity Rule:** Since $X \rightarrow Y$ and $YZ \rightarrow W$ hold, the **Pseudo Transitivity Rule**, rule allows us to substitute Y with X in the second dependency to infer:

- $XZ \rightarrow W$

Armstrong Relation

An Armstrong Relation for a set of functional dependencies F is a relation (table) that perfectly represents F . It must satisfy all functional dependencies in the F^+ Closure and strictly violate all functional dependencies that are NOT in the F^+ Closure. In a given set of dependencies, the size of the minimum Armstrong Relation can be an exponential function of the number of attributes present in the dependency under consideration.

Why Armstrong Axioms Are Considered Sound and Complete?

- **Soundness:** Armstrong's axioms are sound because any functional dependency inferred using them will always be valid and hold true in every relation state that satisfies the original set of dependencies.
- **Completeness:** Armstrong's axioms are complete because applying them repeatedly will generate all possible functional dependencies that can be derived from the original set, ensuring no dependencies are missed.